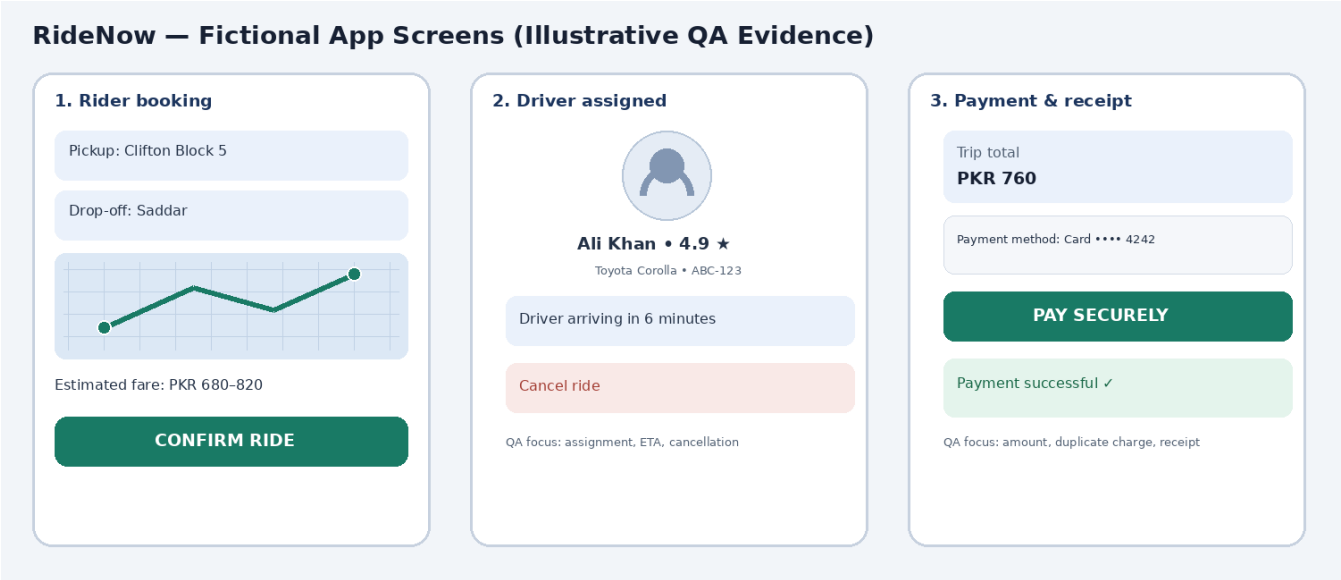
RIDE-HAILING QA ANALYSIS DOCUMENT

Phase 1 — QA Foundations and Manual Testing Mastery

Prepared by: Maryam Masood **Project:** RideNow (fictional ride-hailing app) **Deadline:** 3 days



1. Project Overview

RideNow is a fictional mobile app that lets riders register, set pickup and destination points, view fare estimates, request rides, track assigned drivers, pay, rate trips, and contact support. Drivers can register, manage availability, accept or reject requests, navigate to pickup/drop-off points, and view earnings. An admin portal manages users, trips, pricing, complaints, and driver verification.

Objective: identify testable functional areas, select suitable testing types, clarify requirements before execution, and prioritize risks by likelihood and business impact.

Scope: rider app, driver app, payment integration, notifications, maps/location services, and admin workflows. This report is a planning artifact; no real product has been tested and the screens above are illustrative mockups, not captured production screenshots.

Assumptions: service is available in selected city zones; fares use a base fare plus distance/time and possible surge pricing; payment may be cash or card; live location requires user permission; ride status is updated server-side.

2. Functional Areas Requiring Testing

ID	Functional area	Key checks / example scenarios
FA-01	Registration & login	Valid/invalid phone or email; OTP expiry/resend; password reset; logout; session timeout; duplicate account.
FA-02	Profile & permissions	Edit profile; mandatory fields; location permission allow/deny; privacy and account deletion.
FA-03	Pickup, destination & maps	Search addresses; pin movement; GPS accuracy; service-zone boundaries; route refresh; unavailable location.
FA-04	Fare estimate & promo	Fare calculation; distance/time inputs; surge disclosure; promo validity, expiry, limits, and stacking.
FA-05	Ride request & matching	Request submission; double-tap prevention; driver availability; matching timeout; no-driver path; retry.
FA-06	Driver app & trip lifecycle	Accept/reject; pickup arrival; start/end trip; valid state transitions; GPS updates; connectivity loss.
FA-07	Cancellation & no-show	Rider/driver cancellation rules; fee calculation; cancellation window; reason capture; refund handling.
FA-08	Payments, wallet & receipts	Cash/card; authorization/decline; duplicate charge prevention; wallet balance; refund; receipt totals.
FA-09	Notifications & communication	Push/SMS/in-app events; correct trip and user; delivery delay; masked phone/chat; permission denied.
FA-10	Ratings, feedback & support	Rating limits; duplicate submission; complaint attachment; support ticket status; abusive content controls.
FA-11	Driver onboarding & earnings	Document upload/verification; approval/rejection; availability toggle; commission; payout records.
FA-12	Admin, security & audit	Role permissions; user/driver suspension; trip lookup; pricing changes; audit trail; personal-data access.

Cross-cutting checks: usability/accessibility, localization and currency format, device/OS compatibility, poor-network behavior, data consistency across rider/driver/admin views, and safe handling of personal and payment data.

3. Applicable Testing Types

Testing should be risk-based and layered. The examples below show how each type applies to RideNow; exact execution depends on finalized requirements and available test environments.

Testing type	Category	RideNow example / purpose
Functional testing	Functional	Verify login, booking, matching, cancellation, fare calculation, payment, refunds, and ratings against requirements.
Integration testing	Functional	Check app ↔ backend ↔ maps ↔ payment gateway ↔ notification provider data and error handling.
End-to-end testing	Functional	Complete a trip from sign-in through booking, driver assignment, trip completion, payment, and receipt.
Regression testing	Change-related	After a pricing or booking change, rerun critical booking, payment, cancellation, and trip-state tests.
Smoke testing	Functional / build verification	Quickly verify app launch, login, map load, ride request, and payment sandbox after a new build.
Sanity testing	Change-related	Narrow validation of a specific fix, such as promo-code eligibility or cancellation fee calculation.
Performance/load testing	Non-functional	Simulate peak booking bursts; measure API response times, matching latency, and error rate.
Security & privacy testing	Non-functional	Test access control, session handling, injection risks, sensitive-data exposure, and payment-data handling.
Usability & accessibility	Non-functional	Check readable fare details, clear cancellation steps, screen-reader labels, contrast, and touch target sizes.
Compatibility testing	Non-functional	Verify supported Android/iOS versions, screen sizes, network types, and location-permission states.
Exploratory testing	Technique / cross-cutting	Explore unusual sequences: cancel during matching, lose signal during trip, or reopen app after payment.
API testing	Functional / integration	Validate status codes, schemas, authorization, boundary values, idempotency, and error responses in Postman.

Black-box example: enter pickup/drop-off and verify the fare displayed without inspecting code. **White-box example:** developer-level branch coverage for fare and cancellation-fee rules. **Grey-box example:** submit a ride request while inspecting API responses and database trip status to confirm consistency.

4. Five Ambiguity Questions for Stakeholders

Q1 — Fare calculation

Does the displayed estimate include taxes, tolls, waiting time, and surge pricing? What is the rounding rule, and can the final fare exceed the estimate?

Q2 — Cancellation policy

At what exact trip states can a rider or driver cancel, and when is a cancellation fee charged or waived?

Q3 — Matching behavior

How long should matching continue before timeout, how many drivers are tried, and what should the rider see if no driver accepts?

Q4 — Payment failure

If payment times out after the gateway may have charged the card, how does the app verify the result and prevent duplicate charges?

Q5 — Location/privacy

Is location collected only during an active trip or also in the background, and what happens when location permission is denied or GPS is inaccurate?

5. Risk Assessment & Prioritization

Scoring: Likelihood (L) and Business Impact (I) each use 1–5. Risk score = L × I. High = 15–25; Medium = 8–14; Low = 1–7. Ratings are initial QA estimates and should be reviewed with product, engineering, and operations.

Rank / feature	L	I	Score	Risk rationale & priority test
1. Payments / refunds	4	5	20 High	Duplicate charge, incorrect refund, or payment state mismatch can cause financial loss and complaints. Test idempotency, timeout, decline, retry, refund, and receipt.
2. Booking / matching	4	5	20 High	A rider may be unable to book or may receive multiple assignments. Test repeated taps, concurrency, no-driver, timeout, and state consistency.
3. Fare / surge / promo	4	5	20 High	Incorrect price undermines trust and revenue. Test boundaries, rounding, surge disclosure, expired/stacked promos, and final-fare rules.
4. Trip lifecycle / GPS	3	5	15 High	Wrong pickup, stale tracking, or invalid trip-state transition affects safety and trip completion. Test GPS drift, offline recovery, and state transitions.
5. Cancellation / no-show	3	4	12 Medium	Incorrect fees or refunds create disputes. Test cancellation timing, actor, reason, and payment reversal.
6. Login / account access	3	4	12 Medium	Account takeover or blocked legitimate access harms trust. Test OTP abuse controls, session expiry, reset, and authorization.
7. Notifications	3	3	9 Medium	Late or misdirected updates create confusion. Test delivery delay, correct recipient, duplicate events, and disabled push.
8. Ratings / support	2	3	6 Low	Duplicate ratings or lost complaints reduce service quality. Test limits, persistence, and ticket status.

6. Practical Execution Plan & Evidence

Day 1 — Scope and design: confirm assumptions; ask the five ambiguity questions; map requirements to test scenarios; prepare test data for rider, driver, promo, cash/card, and location-permission states.

Day 2 — Manual execution: execute critical happy paths and negative cases; capture actual results; log defects with ID, environment, build, preconditions, steps, expected/actual result, severity, priority, and evidence. Retest fixes and run targeted regression.

Day 3 — Review and submission: review risk scores with stakeholders; ensure traceability from feature → scenario → result/defect; finalize report and record a concise screen demo.

Suggested defect report example (hypothetical)

Field	Example
Title	[Payment] Retry after timeout may submit a second charge
Precondition	Test account has a valid card in payment sandbox; completed trip is awaiting payment.
Steps	1. Tap Pay. 2. Simulate gateway timeout. 3. Tap Retry. 4. Inspect gateway sandbox events and trip receipt.
Expected	One successful charge maximum; app reconciles the first request before retry and displays one receipt.
Actual	To be recorded during execution; this is a sample defect hypothesis, not a confirmed bug.
Severity / priority	Proposed: High / P1 if duplicate charge is reproducible.

Evidence note: the mock screens on page 1 are original illustrative UI mockups. For a real submission, replace or supplement them with screenshots of your own test artifacts: a test-case sheet, a defect ticket in Jira/Trello, Postman request/response, or an executed scenario with pass/fail status. Do not label mockups as real execution evidence.

7. Conclusion

RideNow should be tested using a risk-based approach, prioritizing payments, booking/matching, fare accuracy, and trip tracking because failures in these areas have the greatest financial, operational, and customer impact. The five stakeholder questions should be resolved before final test-case approval. Testing results must distinguish confirmed defects from assumptions and hypothetical risks.

4–6 minute demo recording outline

0:00–0:30 introduce project and scope; 0:30–1:20 show the functional-area inventory; 1:20–2:10 explain testing types with one black-box, white-box, and grey-box example; 2:10–2:50 review the five ambiguity questions; 2:50–4:10 explain the ranked risk table and why high-risk features come first; 4:10–5:00 show a sample defect report and explain evidence; 5:00–5:30 summarize the conclusion and next steps.